The React.createContext function here creates a React context. The TypeScript generic notation IsVenueContext tells us the expected type of the data in the context, and the argument to the method is a default starting value. We don't have a useful default, since our starting value is coming from the reducer a few lines down, we can start with null.

After making the createContext call, our context is now available for use. First, though, we're going to set up and populate the reducer to put in the context.

React has a hook method called useReducer that is similar to useState and is meant for when you are using the reducer pattern in your React app. Like useState, the useReducer hook causes React to update components when the reducer changes.

The useReducer hook can take two or three arguments. The first argument is always a function that is a reducer, meaning it matches the reducer pattern of taking state and action arguments and returning a state. If you have two arguments, the second argument is the initial state of the reducer. If you pass the hook three arguments, the third argument is a function that returns the initial state of the reducer and takes the second argument to the hook as its argument.

That's a little confusing, so let me put it this way—the following two calls are roughly identical:

```
useReducer(reducer, initialState(arg))
useReducer(reducer, arg, initialState)
```

The difference is that in the first line, initialState is called before the useReducer hook is invoked, and in the second line, initialState is called lazily after the hook is invoked.

In our call to useReducer, we pass it two variables, venueReducer, which is the reducer function we have not seen yet that is defined in the venue_context file, and initialState(props), also defined in the venue_context, which sets up our basic data.

The useReducer hook returns an array of two values: state and dispatch. The state returns the current state of the reducer, and dispatch is a function that takes an